either direction (in figure 17.11, clockwise or counter-clockwise).

But anyway, we don’t mind having two possible global solutions, there is no further action we should take here.

17.2.4 State Transitions

Now it’s time to translate this constraint into code; luckily for us, that’s not too difficult, given how we designed the simulated annealing algorithm in listing 17.1: we need to pass the definition of the function that computes the transition to the next state, and right there, we can always set the first vertex of the sequence to the same value. We also have the cost function figured out, so we are ready to implement all the missing pieces.

Listing 17.2 starts by exploring the cost function, which – as we have discussed – is the sum of the weights of the edges between two adjacent vertices in the permutation. Still, while computing this value we need to be careful to a couple of details: first and foremost, we assume that the input graph has an edge between every pair of vertices; if this isn’t true, we need to check for it and return a special value (for instance, infinity, or any large enough weight) to basically guarantee any solution including missing edges will be naturally discarded. Another alternative could be checking the solutions when transitions are computed, and making sure that those with missing edges between adjacent vertices will be discarded early.